

Lab 2: Pipelining and Superscalar Concepts With SimpleScalar

Computer Architecture Laboratory

1 Objective

In this lab, students use **sim-outorder** to explore pipelined and superscalar processor architectures. Students configure and simulate in-order and out-of-order processor models, compare IPC, and use a pipeline trace to reason about data forwarding. All commands are written for the current repository layout. Run setup commands from the cloned repository root.

```
cd /path/to/SimpleScalar
```

2 Background: The sim-outorder Simulator

sim-outorder is a detailed microarchitectural timing simulator. It models branch prediction, caches, out-of-order execution, functional units, an instruction window, a load-store queue, and in-order commit. Get help for the simulator:

```
./sim-outorder -h
```

3 Setup

Use the PISA build for this lab:

```
make clean
make config-pisa
make
```

Create a workspace:

```
mkdir -p labs
```

Copy the default configuration and the test program:

```
cp config/default.cfg labs/
cp tests-pisa/bin.little/test-math labs/
```

4 Part 1: In-Order Pipelined Processor

4.1 Procedure

Create config_a.cfg:

```
cp labs/default.cfg labs/config_a.cfg
```

Open labs/config_a.cfg in a text editor and set:

```
-decode:width 1
-fetch:ifqsize 1
-lsq:size 8
-ruu:size 8
-issue:width 1
-issue:inorder true
-res:ialu 1
-res:imult 1
-res:memport 1
-res:fpalu 1
-res:fpmult 1
```

Meaning of the parameters:

Parameter	Meaning
-decode:width	Number of instructions decoded per cycle.
-fetch:ifqsize	Instruction fetch queue size.
-lsq:size	Number of memory operations that can be active at once.
-ruu:size	Register update unit size; the main instruction window.
-issue:width	Number of instructions that can be issued per cycle.
-issue:inorder	Forces program-order issue when set to true .
-res:ialu	Number of integer ALUs.
-res:imult	Number of integer multiply/divide units.
-res:memport	Number of memory ports.
-res:fpalu	Number of floating-point ALUs.
-res:fpmult	Number of floating-point multiply/divide units.

Run the simulation:

```
cd labs
../sim-outorder -config config_a.cfg -ptrace config_a.trc 0:1024 -redir:sim sim_configa.out ./test-math
```

View the IPC:

```
grep sim_IPC sim_configa.out
```

View the pipeline trace:

```
../pipeview.pl config_a.trc | less
```

4.2 Questions for Part 1

1. Based on the settings in `config_a.cfg`, describe the processor architecture.
2. What is the final value of `sim_IPC`?
3. Is data forwarding implemented? To answer, examine the pipeline trace for a RAW hazard. Look for a sequence like:

```
ag: sll r2,r16,2  
ah: addu r3,r3,r2
```

Can you find a cycle where `addu` is in the Execute stage while `sll` is in the Write-Back stage?

5 Part 2: Out-of-Order Superscalar Processor

5.1 Procedure

Make sure you are in the `labs/` directory:

```
cd /path/to/SimpleScalar/labs
```

Create `config_b.cfg`:

```
cp config_a.cfg config_b.cfg
```

Open `config_b.cfg` and apply the same modifications as Part 1, with one exception:

```
-issue:inorder false
```

Run the simulation:

```
../sim-outorder -config config_b.cfg -redir:sim sim_configb.out ./test-math
```

View the IPC:

```
grep sim_IPC sim_configb.out
```

5.2 Questions for Part 2

1. Describe the new configuration. What is the key difference compared with Part 1?
2. What is the `sim_IPC` for this configuration?
3. Compare the IPC from Part 1 and Part 2. What does this tell you about the benefit of out-of-order execution for this workload?
4. In an out-of-order processor, the Commit stage ensures results are written back in program order. Why is in-order commit necessary?